Mean-Centered Calibrated KV Quantization

For int8 quality below 5 bit/cell: status, work plan, and the one measured lever

Matthias Kronenberger

Version 0.16 · built 2026-11-11, 12:44 CET · first version 2026-10-25

Abstract

The exact-cell window stores a 4-bit KV cache at 5.29 bit/cell (1,016 MiB at 128K, 36% below `int8`'s 1,584 MiB), but `int8` remains ahead on the statistics that carry weight: the paired 4K mean, the 128K whole segment, p99 and the last 1,024 positions on both texts. The predecessor paper left two bars for a follow-up: opt-in parity at 6.0 bit/cell, and a default change at any rate below 8.25. Neither is a product target — 6.0 sits above the shipped 5.29, and a win at 8.2 saves 0.6% of the cache. This paper asks the follow-up question in product terms: what would a point have to look like that stores at most **5.0 bit/cell** — 6% below the shipped window and 39% below `int8` — and is not behind `int8` on any robust statistic? We fix the target operationally (six criteria, set before the first run), compute the byte budget (0.50 bit/cell for the ring: 4,096 cells in fp16, 7,943 with an `int8` ring, 5,461 at 12 bit, against 6,464 today), locate the remaining error from the predecessor's measurements (the bulk, not the overlay; and immediately outside the window, not only far from it), list six levers with their bit cost and their kill, and give a five-stage falsification ladder with explicit stop conditions. *No number in this paper is a measurement of the 5.0-bit point.* Since version 0.5 the paper carries one lever that is measured rather than proposed: the per-channel offset of the cached K and V rows. Removing it before the rotation — a calibrated table of 3,072 values per side, which is the zero-point of an asymmetric quantizer — is worth 0.17 dB of block error on the key side and 0.36 dB on the value side. End to end it is worth +0.00677 nats on the plain 4-bit arm, and that is where it ends: on the 8-level value table the gain shrinks to +0.00218, and on the 16-level-on-both-sides codec it disappears (−0.00079, not significant), because the level table and the calibration recover the same wasted range and the level table is the cheaper of the two. The arm the plan therefore selects is **16 levels on both sides with the exact-cell window**: on cwe-4k it reaches `int8`'s level (mean KL 0.01689 against 0.01688, a paired difference of +0.00002, $t = -0.02$), with no per-model artifact, no basis discipline in the snapshot and no extra runtime. The 5.0 numbers remain byte-model arithmetic on measured points and are labelled as such.

1 Introduction

The predecessor paper ended with a measured trade-off and an open question [1]. The exact-cell window keeps the most recent S=256 cells and R=64 sink cells of each of the 12 cache-carrying QSA layers in fp16 while the rest of the cache stays in the 4-bit Lloyd format. At 128K it uses 5.29 bit/cell and 1,016 MiB against `int8`'s 8.25 bit/cell and 1,584 MiB, frees 568 MiB for expert weights (416 more resident slots), holds prefill throughput (528 against 526 t/s) and keeps decode inside run-to-run spread. On the fixed cwe corpora it removes 73% of the plain 4-bit base's KL gap to `int8` at 4K and 34–58% of its 128K tail-mean KL. It does not beat `int8`: at 4K `int8` is ahead on the paired mean ($t = -3.55$, 7 of 8 segments), and at 128K it keeps the p99, the last 1,024 positions and the whole segment on both texts.

That paper named two bars for a successor [1]. Bar A asked for opt-in parity within half of the measured gaps at no more than 6.0 bit/cell. Bar B asked for a default change: not behind `int8` on the paired 4K mean, the 128K whole segment or p99 on either text, at any rate below 8.25 bit/cell. Both were written to be honest about what the evidence supports, and both are too weak as a *target*.

Bar A's ceiling sits above today's measured point: an arm at 6.0 bit/cell would be 13% *larger* than the shipped window while merely tying `int8` within a margin. Bar B has no byte ceiling at all, so a win at 8.2 bit/cell saves 0.6% of the cache — a quality result without a product. A target that is worth a branch has to bind both sides at once: a byte ceiling below today's point and a quality floor at `int8`.

This paper sets that target at **5.0 bit/cell**: 6% below the shipped window (5.29) and 39% below `int8` (8.25), with `int8` not ahead on any robust statistic. Five bits is not a round number borrowed from the literature; it is the point where the arithmetic below still leaves a ring large enough to cover the cells the indexer reads, while the bulk has to become better than 4-bit Lloyd to close the rest. The paper's contributions are:

- an operational definition of "better than `int8` in all areas" (six criteria, fixed before the first run, with the statistics that are deliberately excluded and why);
- the byte budget at 5.0 bit/cell, including three ring tiers (fp16, `int8`, 12 bit) and the break-even move;
- the location of the remaining error, read off the predecessor's measurements: the bulk carries it, and the last-1,024 loss sits immediately outside the exact window, not only far from it;
- six levers with their bit cost, their gate and their kill, and a five-stage ladder with explicit stop conditions.

What this paper is not: it reports no new measurement, changes no engine code, and does not claim that 5.0 bit/cell is reachable. Its 5.0 numbers are byte-model arithmetic on

two measured points, and every such number is marked. The paper is a pre-registration: it fixes the target, the byte ceiling and the kills before the first GPU run, so that a later result cannot be fitted to a moving bar. A negative result — the ladder ending without a winner — is an intended outcome and is cheaper than an open-ended sweep.

2 The Measured Point and the Target

2.1 What the predecessor measured

Table 1 collects the numbers this paper builds on. All of them are the predecessor's [1, 31]; none was re-run here. The engine is 0.1.44.1 for the quality and runtime tables and 0.1.47 for the needle pair, on one RX 9060 XT (gfx1200), Windows, ROCm/HIP 7.1, Swift 1.5 IQ3_XXS, `--spec 4`. The KL is teacher-forced against an fp16 KV reference and computed over the reference's top-64 tokens, renormalized.

One row of that table was checked against the merged build, because the numbers below had to be comparable with it: the cwe-4k row coincides to the last digit (`int8` 0.01688, window 0.02090), so the merge moved neither the quality nor the protocol. The runtime rows could not be inherited — the campaign's time values were withdrawn, because they were taken with foreign GPU load — and were measured again (Section 2).

Two readings of Table 1 shape everything that follows. First, the window's deficit is not uniform: it is a whole-segment and tail-quantile deficit, while the tail *mean* favors each arm on one text and reverses sign between texts. Second, the deficit survives even where 256 of the last 1,024 positions are held exactly (the window's S=256): the remaining 768 positions are 4-bit, and `int8` is ahead on that statistic on both texts. A larger exact overlay is the obvious response, and the next section shows why it is not available at five bits.

2.2 How the new numbers were measured

The numbers in Tables 3 and 4 are this paper's own. They were measured after the plan was written, on a re-evaluation of the merged engine (0.1.59.1 — the upstream merge of the quantization branch, so that every arm runs on the build the rest of the project runs on). Four properties of the protocol matter for reading them.

The harness is the predecessor's. One GPU (RX 9060 XT, gfx1200), Windows, ROCm/HIP 7.2, Swift 1.5 IQ3_XXS, `--spec 4`. The quality arms are teacher-forced against the fp16 KV reference over the reference's top-64 tokens, renormalized, exactly as the predecessor scored them, so the two generations of numbers are comparable. The corpus is cwe-4k: 32,746 positions in eight segments.

The gates are effectiveness gates, not only quality gates. Every arm's transcript is hashed. The `int8` arm is byte-identical to the reference run, and an arm that is supposed to change something must produce a *different* hash. This is not a formality: the first run of the calibrated arm produced an identical transcript — the verification path rotated the rows on its own, so the arm did nothing — and a quality table alone would have shown a plausible number for it. A number that does not move because the arm did nothing is the failure mode a hash catches and a KL does not.

The statistics are paired. The comparison of two arms is a paired per-segment difference over the eight segments (the text cancels), with a paired t, and per position a 95 % interval. "Not behind" is only claimed where the interval allows it.

The yardstick carries no bias of its own. `int8` is the anchor every arm is measured against, so it is worth asking whether that anchor drifts. It does not: its KV codec stores an fp16 scale per 64 values and an 8-bit code per value ($\texttt{scale} = \texttt{fp16}(\max|x|/127)$, $\texttt{code} = \text{clamp}(\text{rint}(x/\text{scale}), -127, 127)$), a symmetric grid with round-to-nearest, and measured on the engine's own cells its drift of the per-channel mean is $1.2 \cdot 10^{-4}$ sd on the key side and $4.8 \cdot 10^{-5}$ sd on the value side — three orders of magnitude below the channel bias the data itself carries (0.2673 sd). The clamp is inert: it can only bite if the fp16 scale rounds down by more than $0.5/127$, and the fp16 rounding is at most $4.9 \cdot 10^{-4}$, a factor of eight short; no value in 402 million hit it. What the scale's own rounding does do is shrink the extreme of a group by $|\varepsilon| \cdot \max|x|$ when it rounds down — and the second fp16 rounding, on the product, cancels it almost exactly (the reconstructed extreme is off by $3 \cdot 10^{-7}$ sd). So the gap between the arms and the anchor is noise, not drift, and it has to be explained as noise [32].

The calibration table. The offsets are per layer, per KV head and per channel, with K and V separate: 3,072 values per side, 12 KB in fp16. They are read from a dump of the engine's own rows (12 layers × 32,746 cells × 2 KV heads); the dump holds the rotated rows and the table lives in the channel space, so the transform is undone once — the Walsh–Hadamard is an involution. The table is opt-in through a loader switch, the `int8` path stays byte-identical, and the values are added back on the output path with one vector add per token and head, because the attention weights sum to one. The corrections are subtracted before the rotation and added back after it, in that order.

2.3 The runtime of the same arms

Table 2 measures the same arms on the same build. The measure is the prompt read time of a 32K needle in the predecessor's needle harness: a 32,782-token haystack with the needle at 10 %, 50 % and 90 % depth, one engine at a time, the machine otherwise idle; the decode is nine to ten tokens and negligible, so the number is the prompt read.

Three readings. First, the price of the level table is a **missing fast path**, not a slower codec: the prompt fast path declines the Lloyd modes structurally (it computes $\sum_i q_i(\text{code} - 8)$ in the tensor core and applies the scale afterwards, which a non-affine level table cannot do), so the decode kernel runs instead — and the log shows it, the `int8` arm reading through in 6,144-token chunks while the Lloyd arm stands on the full 30,782 tokens and only then starts working in chunks. The 2.25× is the cost of that fallback, and it is why the plan's level-table lever is a quality lever and not a runtime one. Second, the **calibration is cheap in time**: it costs 2.1 s, or 1.8 %, on the prompt read — a subtraction

Table 1: The measured point this paper starts from. `int8` is the default and the quality reference; the window is `--kv q4lm8v` with `STRATA_KV_WINDOW=1`, S=256, R=64, automatic chunk B=6,144. KL values are mean KL against the fp16 KV reference (top-64, renormalized); lower is better. Δ is window minus `int8`, so a positive value is a window loss. T1/T2 are the two 128K texts. Source: the predecessor's Tables 4, 6 and 7 and its Section 7.3 [1, 31].

Statistic	`int8`	window 256/64	Δ
Rate at 128K (bit/cell)	8.25	5.29	−2.96
KV memory at 128K (MiB)	1,584	1,016	−568 (−36%)
cwe-2k, mean KL	0.00503	0.00702	+0.00199 (t= − 1.92)
cwe-4k, mean KL	0.01688	0.02090	+0.00402 (t= − 3.55; 7/8)
128K T1, whole segment	0.10715	0.13312	+0.02597
128K T1, p99	0.01655	0.02977	+0.01322
128K T1, last 1,024	0.00265	0.00444	+0.00179
128K T1, tail mean (4,096)	0.00959	0.00663	−0.00296 (t= − 0.74)
128K T2, whole segment	0.06435	0.07524	+0.01089
128K T2, p99	0.0165	0.0210	+0.0045
128K T2, tail mean (4,096)	0.00324	0.00437	+0.00113
Prefill (t/s)	526	528	+0.4%
Decode (t/s)	—	−1.0	inside spread
Gates	SHA, 12/12, 6/6+6/6	same	green

Table 2: Prompt read time per 32K needle (three depths, 10/50/90 %, all three found in every arm), one engine at a time on an otherwise idle machine, the same build in every arm. The rate is the prompt tokens per second. The last column is the factor against the `int8` path.

Arm	Read (s)	tok/s	vs. `int8`
`int8`	51.7	596	1.00×
`q4_0`	50.0	616	0.97×
`q4lm8v`	116.4	265	2.25×
+ calibration	118.5	260	2.29×
+ window	116.5	264	2.25×

of 256 offsets per row in the writer and one vector add per token and head in the reader. Third, the **window overlay is free here**: the arm with the exact-cell pool reads the prompt in the same 116.5 s as the arm without it, inside the spread of the three depths.

2.4 What the merged engine adds: the per-channel offset

One lever of this plan has since been measured rather than proposed. The merged engine (0.1.59.1 — the build every arm of the re-evaluation runs on) can subtract a calibrated per-channel offset from the K and V rows before the rotation. The offset is what the channel moments show: the standardized channel mean has median 0.2673 before the rotation, against a sampling expectation of 0.0044 at that token count, and the rotation does not remove it — it turns a fixed offset per channel into a fixed bias in every rotated coordinate. Our block scale is a maximum statistic ($d = \text{amax}/l_{\max}$, the shuffle maximum over the 32 values of a block), so that bias is paid for by every value of the block. Measured on the engine's own cells (the cell dump of 2026-11-10: 12 layers × 32,746 cells × 2 KV heads), removing it cuts the block error by 0.17 dB on the key side and 0.36 dB on the value side — less than the level table (0.30 dB) and the per-block scale (0.31 dB) together, and much less than the 1.10 dB that the level split is worth. The block measurement is the smaller of the two: end to end the same correction is worth +0.00677 nats on the plain 4-bit arm, because the cells that matter for the KL are not the average cell.

On the quality side the same change moves the window arm on cwe-4k from mean KL 0.02090 to 0.01871 against `int8`'s 0.01688: the paired difference falls from +0.00402 to +0.00184 and is no longer statistically significant ($t = -1.85$; the per-position interval $[-0.00429, +0.00062]$ includes zero). The plain 4-bit arm falls from 0.03182 to 0.02505. K1 is therefore two of its three conditions met — the upper bound ($+0.00062 < +0.001$) holds, while the difference stays positive and `int8` still leads in 6 of 8 segments — and the honest reading is that the arm is no longer statistically behind `int8` but not yet level with it. Table 3 is the full comparison, with and without the calibration; Table 4 isolates the calibration itself, paired.

The correction is exact, not an approximation: centering the keys shifts every logit of one query by the same constant, so the softmax does not move, and the values are shifted back by one vector add on the attention output. The offsets must be calibrated per model and per side (K and V differ, and so do the two KV heads); the stability that makes that affordable was measured over a second text and a second length (correlation 0.9489, mean deviation 0.0799 sd). What the level table does to this lever is now measured as well: on the 16-levels-on-both-sides codec the calibration's own effect disappears (−0.00079 nats, the per-position interval $[-0.00341, +0.00183]$), because the level table and the calibration recover the same wasted range and the one that needs no per-model artifact is the cheaper of the two. Two things stay open: the effect at 128K on texts the offsets were not calibrated on, and the question the two measurements leave open between them — the block harness attributes 1.9 dB to the calibration on the 16/16 codec, while the engine

attributes nothing to it there, and that disagreement is not resolved here (Section 2).

Table 3 is the provisional comparison the plan has been waiting for: the same arms with and without the calibration, on one text, one length, one build. Table 4 isolates the calibration itself, paired.

Four readings. First, the calibration is worth something where there is something to win: on the plain 16/8 arm it is +0.00677 nats, significant in the per-position interval and in 8 of 8 segments. Second, its effect shrinks as the arm gets better: +0.00218 on the window arm, where the per-position interval $[-0.00022, +0.00459]$ no longer excludes zero, and −0.00079 on the 16/16 arm, which is nothing — the mean KL is even slightly worse with the calibration (0.01689 against 0.01768), while the p99 and the top-1 agreement are better (0.2410 against 0.2263; 98.89 % against 98.97 %). Third, the level split is the larger and the more reliable of the two: against the window arm, the 16/16 codec is +0.00400 nats better (per-position $[+0.00138, +0.00663]$, $t = +2.99$), and the calibration on top of the 16/8 arm is +0.00218 at best. **The two levers are substitutes** — they recover the same wasted range — and the one that also costs no artifact wins.

A fourth reading comes from the stock row, and it puts the work in proportion. The shipped 4-bit cache sits +0.00843 behind `int8`; the exact-cell window brings that to +0.00402, and the 16/16 codec brings it to +0.00002 — a statistical tie with `int8` (per-position $[-0.00240, +0.00237]$, $t = -0.01$), with a p99 of 0.2410 against `int8`'s 0.1888. The same table also shows what the plan did not expect: the level table *without* the window is worse than the stock grid (+0.01495 against +0.00843), because the level-table family measured here carries the 8-level value table, and that probe costs 1.10 dB of block error for no byte.

The product reading of the rows together is therefore: **the 16/16 codec with the window, without the calibration**. It reaches `int8`'s level on this corpus, it needs no per-model artifact, no basis discipline in the snapshot and no extra runtime, and it is the arm the ladder's first step selects. The calibration stays a lever for the 8-level-value probe and for any codec that wastes range — a useful answer, but not the one this plan started from.

The two statistics on which the calibrated 16/16 arm looks better — the p99 (0.2263 against 0.2410) and the top-1 agreement (98.97 % against 98.89 %) — are each a quantile or a count without a paired test, and both were put to one. The top-1 difference is not significant: the two arms disagree on 346 of 32,746 positions, 186 in favour of the calibrated arm and 160 against it, an exact McNemar $p = 0.18$. On a tail defined once and independently of the arms — the 1 % of positions where `int8`'s KL is largest — the uncalibrated arm is the better one (mean KL 1.080 against 1.211), while both are far ahead of `int8` (1.445). So the reading that survives is the one the paired mean already gave, and the p99 has to be re-checked on the 128K texts, where it is a criterion of its own (K3), with the same paired tools.

2.5 The target, fixed before the first run

The target has six criteria. All are checked in the order given; a criterion that fails ends the arm. The statistics are the predecessor's, so the comparison is paired and same-harness.

Three choices in Table 5 need a reason. The tail mean is excluded because its sign is not stable across the two 128K texts (Table 1): a statistic that favors each arm once cannot carry a target, and the predecessor already used the p99, the last 1,024 and the whole segment for its robust reading. cwe-2k is excluded because the window is already statistically indistinguishable from `int8` there ($t = -1.92$, window ahead in 2 of 8 segments); it would be a criterion no arm can fail, not a test. Equality is accepted on every statistic because the byte rate is 39% lower: at five bits a tie is a product, while at 8.25 it is not.

K5's 10% is the only threshold without a measured basis; it is a choice, set because the predecessor's window is at +0.4% and because the one measured small-ring configuration (B=512) pays 80% of prefill. The threshold exists so that a quality win cannot be bought with an unmeasured prefill collapse. K6 keeps the predecessor's gates and adds the two open review gates, because a default claim needs the full cell-to-row parity and the other two backends.

Finally, what "all areas" cannot mean here. The harness measures single-segment, teacher-forced KL on one task family, on one GPU, one operating system, one weight set. It does not measure multi-turn growth, concurrency, mid-segment KL against the other arms, or a second task family. The target is therefore a claim about the statistics in Table 5 and nothing wider; Section 8 states the bound.

3 The Byte Budget at Five Bits

The predecessor's byte model is one line [1]:

$$\text{bits/cell} \;\approx\; 4.50 \;+\; \frac{16\,(R+S+B)}{L}, \tag{1}$$

where L is the context length in cells, the 4.50 is the 4-bit base (1,024 values per cell and layer in 32 blocks of 18 bytes = 576 B), and the second term is the resident ring: the exact side copy costs 16 bit/cell *in addition* to the 4-bit row it duplicates, which is why the measured point is $4.50 + 16 \times 6{,}464/131{,}072 = 5.289$.

The equation is the whole argument for the ceiling. At 5.0 bit/cell the ring may spend 0.50 bit/cell, which at L=131,072 is 4,096 cells in fp16. With S+R=320 that leaves $B \le 3{,}776$, against 6,144 today: the fp16 ring has to shrink by 37% (6,464 to 4,096 cells) while the quality floor rises. The ring does not have to be fp16, though. An `int8` side copy costs 8.25 bit/cell (the default's 8 bits plus its 0.25-bit scale share), which buys 7,943 cells — 23% more than today — and a 12-bit ring buys 5,461. Table 6 lists the three tiers.

The tier choice is the paper's central hypothesis, and it is not a byte trick. `int8` is the quality reference, not fp16: a ring that holds its cells at `int8` precision makes the ring region *equal* to `int8` by construction, and spends the remaining

Table 3: Provisional quality comparison, with and without the mean-centered calibration. cwe-4k, 32,746 positions, top-64 KL against the fp16 reference, the merged engine (0.1.59.1), the same ids and the same build for every arm; the offsets come from a calibration run on this same text. The comparison is therefore *in-sample*: it says what the calibration does to the arms, not that the offsets transfer to texts they were not calibrated on (the 128K out-of-sample run is open, Section 2). The last three columns are the paired difference against `int8` over the eight segments; a positive value means the arm is behind `int8`. The `q4_0` row is the stock 4-bit cache (the uniform grid after the rotation, no window, no calibration). The `q4lm8v` rows are the level-table family with its 8-level value table — a measurement probe that saves no byte — and the `q4lm32w` rows are the product codec with 16 levels on both sides.

Arm	Mean KL	Top-1	KL p99	PPL	ΔKL vs `int8`	t	`int8` ahead
`int8`	0.01688	99.05 %	0.1888	2.860	—	—	—
`q4_0` (stock)	0.02530	98.71 %	0.3955	2.863	+0.00843	−3.99	8/8
`q4lm8v`	0.03182	98.40 %	0.5785	2.888	+0.01495	−4.58	8/8
+ calibration	0.02505	98.75 %	0.4127	2.871	+0.00818	−3.95	8/8
`q4lm8v` + window	0.02090	98.84 %	0.2811	2.858	+0.00402	−3.55	7/8
+ calibration	0.01871	98.87 %	0.2665	2.862	+0.00184	−1.85	6/8
`q4lm32w` (16/16)	**0.01689**	98.89 %	0.2410	2.852	+0.00002	−0.02	5/8
+ calibration	0.01768	98.97 %	0.2263	2.856	+0.00081	−0.99	5/8

Table 4: The calibration alone, paired: the same arm with and without the offset, the same positions, one comparison pass with the fp16 reference. The value is the improvement (uncalibrated minus calibrated) as the segment mean over the eight segments; t is the paired t on that mean, the next column counts the segments in which the uncalibrated arm is worse, and the interval is the 95 % interval of the per-position difference — the interval that decides, because a segment mean can be carried by one segment.

Arm	Gain	t	Segs.	95 % interval
plain (16/8)	+0.00677	+3.81	8/8	[+0.00350, +0.01003]
+ window (16/8)	+0.00218	+2.88	6/8	[−0.00022, +0.00459]
16/16 (product)	−0.00079	−0.69	5/8	[−0.00341, +0.00183]

quality budget on the bulk. If the measured deficit is concentrated in the ring region — which the last-1,024 statistic suggests (Section 4) — then an `int8` ring attacks exactly the cells that carry the loss, at 8.25 rather than 16 bits. The price is that the ring is no longer exact: a cell that leaves the ring is quantized, and the fault rule of the A-series (a promised exact cell that is no longer present) has to be restated for a tiered ring (Section 5).

Two consequences follow from Table 6. The break-even against `int8` moves from about 27,600 cells (6,464-ring) to about 17,500 (4,096-ring), so a 5.0 point is cheaper than `int8` from roughly 17.5K cells upward — at 24K it is already cheaper, and setup's 32K offer stays valid and gains margin. And the product arithmetic: linear between the two measured points, 5.0 bit/cell is about 960 MiB at 128K — 624 MiB (39%) below `int8` and 56 MiB (6%) below today's window — and about 457 more expert slots than `int8` at 416 slots per 568 MiB. The slot figure is model- and quantization-dependent (Swift 1.5 IQ3_XXS, RX 9060 XT); it is arithmetic, not a measurement.

The gain is memory and slots, not speed. The predecessor's window is at prefill parity, and the smaller ring could cost prefill: the only measured small-ring arm (B=512) ran at 105 against 528 t/s. That is why K5 exists, and why a 5.0 arm that wins on quality but loses more than 10% of prefill is dead in this plan.

One more term belongs in the budget: a per-channel scale before the rotation (lever L3) costs one fp16 value per 128 cells on the K half, $16/128 \times 0.5 \approx 0.06$ bit/cell. At 5.0 that leaves 0.44 bit/cell for the ring — 3,604 fp16 cells or 6,990 `int8` cells — so the levers compete for the same 0.50 and cannot be priced independently.

4 Where the Error Sits

The predecessor's measurements answer, partially, where the remaining error lives, and one channel statistic measured later (the S2 probe of 2026-10-27, in-repo) adds a fifth. Five facts matter for the plan; none is a measurement of the 5.0-bit point.

A larger fp16 ring is arithmetically closed. Under a 6.0 ceiling, 1.5 bit/cell of headroom buys at most 9.4% of the cells (1.5/16, about 12,300 cells at 128K); at 5.0 it buys 3.1% (4,096). The measured 512/128 arm doubles the exact overlay at 4K (320 to 640 positions) and ties 256/64 (t= + 0.02) while costing 7 MiB more at 128K and a slower decode. Spending the ceiling on a bigger fp16 ring is therefore not a route: the bytes are not there, and the one measured doubling bought nothing.

The loss sits next to the window, not only far from it. The last-1,024 statistic is the sharpest evidence. Of those 1,024 positions, 256 are inside the window and stored exact (read exact when the indexer selects them); the other 768 are 4-bit. `int8` wins that statistic on both texts (0.00265 against 0.00444 on T1). So the deficit is produced by cells *immediately* outside the exact overlay — cells that a tiered ring of the same size would cover better, because an `int8` ring cell is closer to `int8` than a 4-bit cell is. This is a static reading of the design plus a measured statistic, not a measurement of the tiered ring.

The tail mean cannot carry a target. It favors the window on T1 (0.00663 against 0.00959, t= − 0.74) and `int8` on T2 (0.00324 against 0.00437). The sign flips between two

Table 5: The 5.0-bit target. "Not behind" means the arm's value is less than or equal to `int8`'s; a tie is not a loss. Anchors are the measured window values from Table 1 and are shown for scale, not as thresholds. K1–K3 are the quality floor, K4 the byte ceiling, K5 the runtime bound, K6 the correctness gates. C1 (full cell-to-row parity) and C2 (CUDA/SYCL builds) are the two review gates still open in the predecessor.

#	Criterion	Anchor (window today)
K1	cwe-4k paired mean KL (arm − `int8`) ≤ 0, upper 95% bound < +0.001 nats, and `int8` ahead in at most 4 of 8 segments	+0.00402 (window), +0.00184 (with the offset), +0.00002 (16/16 codec, $t = -0.02$)
K2	cwe-128k whole-segment KL ≤ `int8` on *both* texts	+0.02597 (T1), +0.01089 (T2)
K3	cwe-128k p99 *and* last 1,024 ≤ `int8` on both texts	p99 +0.01322/+0.00450; last 1,024: 0.00444 vs 0.00265
K4	effective rate at 128K ≤ 5.0 bit/cell (byte model written down before the run)	5.29, 39% below `int8`
K5	prefill at most 10% below `int8` (526 t/s); decode inside run-to-run spread	528 vs 526 t/s; B=512 costs −80%
K6	`int8` dump-logits SHA unchanged; parity 12/12; needles 6/6 + 6/6; setup suite; plus C1 and C2	all green today except C1/C2

Table 6: The ring budget at 5.0 bit/cell (L=131,072, S+R=320). The side copy is additional to the 4-bit row. The 12-bit row is priced at 12.0 bit/cell; with `int8`'s 0.25-bit scale share it would be 5,349 cells. The two lower tiers are arithmetic, not formats: neither exists in the engine yet.

Ring tier	bit/cell	R+S+B	B	vs. today
fp16 (today)	16.0	4,096	3,776	−37%
`int8`	8.25	7,943	7,623	+23%
12 bit	12.0	5,461	5,141	−15%
today's point	16.0	6,464	6,144	—

texts from the same generator, so the target uses the p99, the last 1,024 and the whole segment instead. A 5.0 arm that wins the tail mean on one text has won nothing.

2K is saturated. The window is already indistinguishable from `int8` at 2K ($t = -1.92$, window ahead in 2 of 8 segments). A target statistic that no arm can fail is not a test; 2K is reported, not used.

A fixed per-channel offset rides along with the rotation. The channel statistics measured before and after the transform (the S2 probe of 2026-10-27, in-repo) carry a mean that the plan had not read yet. The standardized channel mean |mean|/sd has median 0.2673 before the transform and 0.2825 after it, against 0.0044 as the sampling expectation at this token count: the offset is real, and the transform does not remove it — it turns a fixed per-channel offset into a fixed bias in every rotated coordinate, because the same offsets add up with the Hadamard signs. The offsets are channel-specific rather than a common mode (the mean of the channel means is 0.007 to 0.068 of the typical channel standard deviation, layer by layer). This matters here because the block scale is a maximum statistic, $d = \text{amax}/l_{\max}$, with amax the shuffle maximum over the 32 values of a block: a fixed bias inflates that maximum, and the inflation is paid in resolution by every value of the block. Centering the keys before the transform is exactly preserving — $\langle q, m \rangle$ shifts every logit of one query by the same constant, so the softmax does not move, and centering the values shifts the output by m, because the attention weights sum to one — and it is the lever the 2026 literature opens [17]. From the moments alone the block scale looks 15% to 30% too large, which would be one to two decibels of mean-squared error; measured on the engine's own cells, with the offsets the engine itself uses, the effect is 0.17 dB on the key side and 0.36 dB on the value side — an order of magnitude less than the estimate from the moments, and the reason a plan should measure the estimate. (An earlier reading of this measurement reported 1.9 dB. That column is **not reproducible**: neither of the harness's two centering modes produces it, so its provenance could not be established and the number is withdrawn [33]. The end-to-end numbers of this paper come from the engine and are unaffected.) The offset is stable across a second text and a second length (correlation 0.9489, mean deviation 0.0799 sd), so the table it needs is a per-model artifact rather than a per-run one.

Where the bias comes from: the data, not the transform. A fixed per-channel offset is a fixed vector in the transform space, and an exactly orthogonal transform maps it back exactly — so a bias that *vanished* under the back-transformation could only be an artifact of the transform's arithmetic, not of the data. The two halves of that question are measured separately here: the channel probe measures the key means *before* the rotation, and the cell dump holds the *rotated* rows, so un-rotating the dump's mean and comparing it with the probe's is a direct test. It comes out at a ratio of **1.000** in every layer and every head: the bias in the transform space is exactly the image of a real bias in the channel space, and the round trip $H(H(y)) - y$ is exactly zero on the dumped cells. The bias is the data's [32].

The applied literature reads it the same way and goes further: the post-Hadamard channels of a trained model *still differ in mean*, and the centering is therefore composed with

the transform rather than kept outside it [10]; a KV-cache rotation orders its channels by a per-channel statistic measured *after* the rotation [9], which only makes sense if that structure is real and stable. The numerics of the transform are a separate matter: the error of a fast Walsh–Hadamard in reduced precision is a rounding error, reducible with compensated summation, not a systematic bias. What *is* a real trap is the opposite of the artifact hypothesis — offsets measured in the wrong basis. A bias is basis-dependent, and the public fork that applies the same correction to the KV cache carries a guard that refuses a bias measured in the wrong basis [20]; the measurement error that this paper had to correct was of exactly that kind, an offset set measured on the key side only and merged over heads, applied to both sides.

The shape of the error: a fat tail, and where it sits. Not only where the error sits but how it is distributed matters at 128K, and the distribution is not Gaussian. Measured on the 128K text against the fp16 reference, the per-position KL of the `int8` arm has an excess kurtosis of 211, its p99 is 25.8 times its mean, and 99 % of the positions are essentially perfect (median 0.00085) while 0.76 % of them carry 74 % of the error. The tail is not noise: it is discrete. Decoding the prompt with the model's own tokenizer and locating the heavy positions against the text shows that 845 of the top-1,000 positions are occurrences of the single most frequent coded word (9,928 occurrences in the text), and the next most frequent words take the next places — the very words whose frequency the task asks the model to track. The model has to keep a counter for each of them in its KV cache, and at every occurrence of the most frequent one that counter is at its most fragile, which is where the quantized and the fp16 models disagree. So the fat tail is a property of the task and the context length, not of the codec: it is already there at `int8`, and it grows with the length (p99 over mean 11.2 at 4K, 25.8 at 128K).

Two consequences for how this paper reads its own numbers. The mean KL at 128K is a statement about about 2,400 positions (1.8 % of the corpus), not about the corpus; the p99 and a paired tail are the statistics that describe a codec, and the ladder's criteria were written that way before this was measured. And a published p99 for this corpus that reads a hundred times smaller would have made the same distribution look mild — which is a reason to prefer a re-measured number to a remembered one.

And at 128K the 4K tie does not survive. The table above is a 4K measurement, and 4K is where this plan's anchors live. The 128K texts are a different corpus with a different length, and there the same arm falls behind `int8` — and not by a hair. Measured on the first 128K text against its own fp16 reference (131,060 positions, the same build, the same ids file):

The gap to `int8` is +0.00741 nats on the mean ($t = +4.80$, the interval $[+0.00439, +0.01044]$ excludes zero) and it is wider on the p99 (2.0583 against 1.6615): on this corpus the 4-bit bulk loses to the 8-bit anchor in the long context, and the deficit grows with the length instead of shrinking — the same curve the predecessor measured for its window arm (+0.00402 at 4K against +0.02597 at 128K). The tail column shows the other half of the picture: at the positions where `int8` breaks down, the 4-bit arm is the closer one (3.42 against 5.25), so the two codecs trade spike against spread, and at 128K that trade is the wrong way round. So the 5.0-bit target — five bits per cell and not behind `int8` — is **not** reached by this arm on this corpus, and the reason is the bulk codec in the long context, not the overlay: the overlay is what buys the 5.29 against 8.25 bit per cell.

Table 7: At 128K. The product arm (16 levels on both sides, with the exact-cell window) against the `int8` anchor, fp16 as the reference, 131,060 positions. The tail is the 1 % of positions where `int8` is worst — defined once, arm-independently. The 4K measurement of the same arm was a tie (+0.00002, $t = -0.02$); here it is behind.

Arm	Mean KL	p99	Top-1	Tail mean
`int8`	0.06435	1.6615	98.45 %	5.24581
`q4lm32w` (16/16)	0.07176	2.0583	98.30 %	**3.42218**

One thing this measurement settles, and it is worth settling: the heavy positions are largely the same for both codecs. The rank correlation of the per-position KL between the two arms is 0.848 and half of the top-1,000 positions coincide, while the shape of the distribution is nearly identical (excess kurtosis 211 against 192). The fat tail belongs to the text and the task; the codecs differ in how much mass, and how sharp a spike, they put into it.

And a third source sits inside the bulk: a fixed per-channel offset. The deficit is not only a coverage question. The bulk's own codec spends part of its range on something that is not signal: the cached K and V rows carry a fixed per-channel offset, and the scale is a maximum statistic, so that offset inflates the scale of every block and is paid for by every value in it. The mechanism is worth stating precisely, because it is easy to get wrong: the offset does **not** make the codec drift (the measured drift of the 4-bit codec is as small as the anchor's, $1.4 \cdot 10^{-4}$ sd), it makes the codec **noisier** — the block's maximum grows by the bias's share, the step grows with it, and the quantization noise grows with the step. Removing the offset before the rotation is worth 0.17 dB of block error on the key side and 0.36 dB on the value side (Section 2) — less than the level table and the per-block scale together — and end to end +0.00677 nats on the plain 4-bit arm. This is the one lever of this paper that is measured rather than proposed, and it is orthogonal to the window: the overlay buys exact cells, the offset buys back the range the bulk wastes. The same measurement, however, shows its limit: the offset and the level table buy back the same range, so on the 16-level-on-both-sides codec the offset has nothing left to buy (Table 3).

Together these facts point one way: the bulk must become

a better code than 4-bit Lloyd — and stop spending range on a fixed offset — and the ring must become cheaper per protected cell. Neither alone reaches 5.0: a smaller fp16 ring loses the statistics `int8` wins, and a better bulk without the byte side stays above the ceiling.

The predecessor also closed five doors, and this plan does not reopen them [36]: clipping after the Walsh–Hadamard (worse 2K KL), Hadamard on/off as a quality arm (a tie on the 16-level base at 2K and 4K), the K-versus-V bit contrast ($t = +0.05$), a 3-bit base without the window (worse), and R=8 as a sink count (too small). The published direction that keys keep the bits [26] stands as its authors' result on their models; it is not a reason to rerun a contrast this model has already failed.

5 Six Levers

Table 8 lists the levers in the order the ladder tests them. Each has a bit cost that comes out of the same 0.50, a gate that must pass before it is built, and a kill. None is a result.

L1 and L2: the code inside the block. The 4-bit block holds K at 16 levels and V at 8. Giving V 16 levels costs no bytes and is the cheapest quality experiment in the plan; a tie kills it, and a tie is the likely outcome given that the predecessor's K-versus-V contrast did not support an asymmetry. A different 16-level table (L2) is the same experiment with a different codebook and the same kill. Neither touches the byte budget, which is why they come first: if the bulk cannot be improved inside 4 bits, the plan's remaining levers all cost bytes. The block-level measurement of this paper answers the first half of the question: the 8-level value ladder is a measurement probe — a 3-bit codebook inside 4-bit bytes — so it saves no byte, and it costs 1.10 dB of block error against the 16-level ladder. The product codec therefore carries 16 levels on both sides, which makes "give V 16 levels" a correction to the shipped mode rather than an experiment; the open part of L2 is the codebook itself. The engine agrees with the block measurement, and the agreement is larger than the block measurement alone would have suggested: against the shipped window arm the 16/16 codec is +0.00400 nats better (per-position [+0.00138, +0.00663], $t = +2.99$), and it reaches `int8`'s level (Section 2). The runtime belongs to the same paragraph, because it is the price of the level table as such and not of the 8-level variant: the level table is what the prompt fast path declines (Section 2), and the arm pays 2.25× the prompt read time for it (Table 2). The decision taken here is quality first — the table stays, the fast path for it is a separate project — but the price is named, because a plan that hides it cannot be weighed.

L5: the selection-following pool, and why it is not a build but a count. The overlay's ring is positional: the exact twin of a cell is written when the cell is written, and it lands in the row the position dictates, while the indexer's selection — which cells the attention actually reads — is only known afterwards, when the cell already sits quantized in the bulk. A pool that followed the selection would therefore have to *predict* it at write time, and its exactness promise would become a promise about a guess rather than about a rule. Two measurements decide whether that is worth anything at all, and both are counters rather than kernels: how fast the union of the selections grows (the pool holds 4,096 cells, the ring's price at 0.50 bit per cell), and whether the positions that carry the 128K error are among the selections at all — the heavy positions are the occurrences of the most frequent coded word, spread over the whole text, and a pool cannot protect what the indexer never reads. The predecessor's own counter says the positional ring covers 8.55 % of the selections, so 91.45 % of the reads are 4-bit today; if the union of the selections outgrows the pool, or if the heavy positions are not read, the lever is closed arithmetically and the 5.0-bit mark stays what it is — a byte question about the ring's tier, not a quality question about its contents.

L3: the channel axis before the rotation. The writer quantizes after the Walsh–Hadamard, which mixes the 64 rotated with the 192 unrotated dimensions. KIVI's per-channel key quantization assumes channels survive to the quantizer [4]; here they do not, unless the scale is applied before the transform. The lever is a per-channel scale on the K half before the transform, at 0.06 bit/cell. Its gate is a measurement, not an argument: the second moments of the K channels before and after the transform, split into the two slices. If the pre-rotation channels are not the heavier tail, the lever is not built. Per-channel scales after the transform are not this lever — the transform has already mixed the channels. The gate ran on 2026-10-27 and killed the tail half: the pre-rotation channels are not the heavier tail (mean excess kurtosis −0.18 before against −0.14 after, mean crest 3.72 before against 3.93 after), so a per-channel *scale* is not built. The same probe carries a mean the plan had not read, and that half is not killed: the standardized channel mean is 0.2673 (median) against a sampling expectation of 0.0044, and the transform spreads it into a fixed bias (Section 4). A per-channel *offset* before the transform is therefore the surviving form of this axis — exact, cheap, and measurable in the block harness before anything is built. It has since been built and measured (Section 2): 0.17 dB of block error on the key side and 0.36 dB on the value side, and end to end +0.00677 nats on the plain 4-bit arm (the window arm from +0.00402 to +0.00184 against `int8`, no longer significant) — but nothing on the 16-level-on-both-sides codec, where the level table has already recovered the range. Its price is a calibrated table (3,072 values per side, 12 KB per model), a basis discipline (a snapshot of a centered state must not restore into an uncentered one) and a second, still open, question: whether the offsets hold at 128K on texts they were not calibrated on.

L4: the ring tier. This is the byte lever, and the one the paper's hypothesis rests on. Three tiers are priced in Table 6; the ladder measures them as three byte variants of the same

Table 8: The six levers. "Cost" is the side-copy budget each one consumes at 5.0 bit/cell; the base's 4.50 is fixed. A lever with no cost changes only the code inside the existing 4-bit block. Gates refer to the ladder stages of Table 9.

Lever	What changes	Cost	Kill
L1 level layout	16 levels on K and V instead of 16/8, same 4-bit block (A2)	0	tie at 4K
L2 level table	a different 16-level table for the same block (A5)	0	no K1 at 4K
L3 per-channel K	one scale per 128 K cells, before the Walsh–Hadamard (A1)	0.06	no heavier pre-rotation tail (S2)
L4 ring tier	`int8` or 12-bit side copy instead of fp16; geometry from the knee sweep (B1)	−7.75 or −4.0 vs. fp16	no K1–K5 in any tier
L5 selection-following pool	exact bytes on cells the indexer returned, every cell still addressable at 4 bit (E3)	0 (same 16/cell)	fewer than half of selections in $R{+}S$ (S1)
L6 residual correction	a low-rank or sparse residual on the 4-bit bulk (GEAR form)	residual bits	dense residual (E4)

bulk winner. The `int8` tier has an engineering argument beyond its arithmetic: the engine already writes and reads `int8` KV for the default, so the tier reuses a code path that is measured and gated, while a 12-bit tier is a new format with its own writer, reader and parity suite. Against that stands the fault rule: today's window promises exact cells, and a tiered ring promises *near-exact* cells. The rule has to be restated — a cell that leaves the ring is quantized to the bulk, never silently dropped — and the parity test (C1) has to cover the tier boundary. The knee sweep (B1) belongs here too: it can redistribute S and R, but it cannot create bytes, so it is a refinement of the tier, not a way around the ceiling.

L5: spend the exact bytes where the indexer looks. The predecessor's counter question is whether the positional overlay covers the cells the indexer actually selects. If fewer than half of the selections fall in $R{+}S$, the positional overlay is in the wrong place, and the alternative is a pool that follows the selection: every cell stays addressable at 4 bits, and the higher-precision copy goes to cells the indexer returned. The selected set changes per query, so the pool is a union or the last selection, and a promised cell that has left the pool is the same fault as a dropped cell. This lever does not start until the counter says the overlay is misplaced; if the counter says the overlay is right, L5 is dead and L4 carries the plan.

L6: the residual. A low-rank or sparse residual on top of the 4-bit bulk is the GEAR form [6]. It is last because its cost is not known before the residual's shape is measured: a dense residual costs bits everywhere, a sparse one costs a mask plus a few values. The gate is therefore the residual's shape on the existing 4K pair — dense kills the lever, sparse prices it. L6 is the only lever that can improve cells the ring does not cover without changing the ring, which is why it stays on the list even though it is the least specified.

6 The Ladder

The ladder has five stages. Each stage asks one question, and the next stage starts only if the previous stage's kill did not fire. The risk the ladder manages is not that an arm fails — failure is the intended outcome — but that the engine's default or the `int8` SHA moves while the plan is being tested. Every stage is opt-in, and every stage writes its byte model down before it runs.

The stop conditions are part of the plan, not an afterthought:

- S0 flips the ranking or the sign count: the bar is undefined; no engine work until the 128K reading exists under the same definition.
- S3 finds no arm reaching K1 at the weaker 6.0 bar: the bulk is not the lever, and the branch ends with a documented negative result. No 128K run is started.
- S4's three variants all miss K1–K5: 5.0 is not reachable with these six levers. The branch either ships the 6.0 opt-in fallback (parity within half of today's gaps at $\leq$ 6.0 bit/cell) or reports the negative result; it does not start a sweep.
- A variant loses more than 10% of prefill: dead, regardless of quality.
- Any stage moves the `int8` SHA or fails parity: stop and fix, no quality claim.

The budget is deliberately small: at most three 4K arms in S3, one 128K block of three variants by two texts in S4, and no 128K sweep. If S3 kills, the whole plan costs one counter run, one debug statistic and three 4K arms — less than a single 128K sweep.

7 Relation to Prior Work

The levers are not new mechanisms; they are the mechanisms that survive this model's geometry, and each maps to a published line of work. The mapping is the honest form of a related-work section for a plan: it says which paper each lever borrows from, and what that paper measured on its own models, not on this one.

Per-channel key scales are KIVI's axis [4]; KIVI pairs them with per-token value scales and a 2-bit budget plus an fp16 residual window, and reports negligible accuracy loss on its models. Here the rotation sits between the channels and the quantizer, which is why L3 is gated on a moment measurement rather than ported. KVQuant's per-channel pre-RoPE treatment and sparse outliers [5] are the same axis in a different placement; this cache rotates 64 of 256 dimensions and

Table 9: The ladder. S0 and S1 are offline or a counter; S3 costs 4K GPU time for at most three arms; S4 is the only 128K block (three byte variants, two texts). A stage that is killed ends the branch with a documented negative result rather than a wider sweep.

Stage	Question	Kill	Cost
S0	Is the bar itself stable? Rescore one existing 4K pair under a second KL definition (top-256 or full vocabulary) on the same positions, signs reported.	ranking or sign flips ⇒ no kernel until 128K is reread under the same definition; K1/K2 undefined	scorer only
S1	Do the indexer's selections fall in R+S? Count, on one 128K prefill, the share in R, in S, in neither, per QSA layer; plus the 4K residual's shape (E4).	$<$ half in R+S ⇒ the positional overlay is misplaced: L5 becomes the candidate and B1 does not run; dense residual ⇒ no L6	one counter run
S2	Is the channel axis heavier *before* the rotation? Second moments of the K channels before and after the transform, split into the 64 rotated and 192 unrotated dimensions.	no heavier pre-rotation tail ⇒ L3 not built	debug statistic, `int8` untouched
S3	Does the cheap bulk tool suffice? 4K, paired against `int8`: (a) 16/16, (b) alternative table, (c) L3 if S2 survived. Each arm reports K1, K4 and bit/cell.	no arm reaches K1 at $\leq$ 6.0 bit (the weaker bar, so not at 5.0 either) ⇒ the bulk is not the lever; no 128K, back to S1 or stop	$\leq$ 3 4K arms
S4	Which 5.0 composition holds? The S3 winner as bulk, three byte variants at 128K, both texts, all K statistics, byte model first: (a) fp16 ring, R+S+B=4,096; (b) `int8` ring, 7,943; (c) 12-bit ring, 5,461.	no variant meets K1–K5 ⇒ 5.0 is not reachable with these levers: fall back to the 6.0 opt-in bar or publish the negative result; prefill $<$ 90% of `int8` kills the variant even if quality passes	one 128K block

normalizes after RoPE, so a pre-RoPE port would quantize a different tensor. GEAR's residual [6] is L6; its near-lossless claim rests on a low-rank plus sparse residual, and the sparse part is the part E4 would have to find here.

The exact overlay itself is the literature's sliding-window and sink protection: StreamingLLM's sinks [3], SKVQ's sliding window [7], IntactKV's pivot tokens [8], RotateKV's sink handling [9]. The predecessor implements that mechanism on the cells this model reads; L4 changes the precision of the protected cells, not their selection. The selection-following variant (L5) is the token/precision trade-off of the top-k line [28], with the indexer's selection left untouched because replacing it would change the model. Layer-wise mixed precision [27] is not a lever here: only 12 of 48 layers carry this cache, and no per-layer error signal exists yet. Learned rotations and rewritten projections [12, 13, 24] are a different model, not a cache flag. The system-aware 4-bit formats [25] are the same family as Lloyd plus Walsh–Hadamard; the predecessor's on/off ablation was a tie at 2K and 4K, so L1/L2 test the code, not the transform.

Two deployed anchors bound the claim. `llama.cpp`'s `q4_0`/`q8_0` KV types [29] are the field's version of the same trade-off, measured on document perplexity, not on this cwe protocol; the predecessor's `q4_0` anchor is a different harness and is not comparable. FP8 [30] is not an attention kernel on this GPU, which is why the two-times point here is `int8` and the frontier is 4 bit.

The offset is the zero-point of an asymmetric quantizer. Removing a fixed per-channel offset before the rotation is a known operation: the Hadamard spreads an outlier into a constant offset that an asymmetric quantizer can absorb in its zero point [18], and the output-aware rotation work centers the keys for the same reason — a shift that leaves the softmax untouched [17]. A public `llama.cpp` fork carries the same correction for the KV cache, together with a guard that refuses a bias measured in the wrong basis: a bias is basis-dependent, and applying it in another basis measurably degrades the quantization instead of improving it [20]. That guard is the design constraint this plan inherits: the offsets are calibrated in the channel space, subtracted before the rotation, and a state carries its basis.

The rotation's job, and what the 2026 literature adds. The Walsh–Hadamard this cache applies before writing belongs to the rotation family that the quantization literature uses to make outliers harmless [12, 9]: it spreads a channel's energy over the 256 coordinates, so that no block maximum has to cover a few large channels. What that literature calls the randomized Hadamard transform is the Hadamard with a random sign diagonal [14] — not the fast randomized Hadamard transform with a random permutation, which is what a Haar approximation would need, while the 2026 theory work shows that the sign-only version suffices [19]; the adaptive variants re-order the channels towards the outliers instead [9], which needs offline statistics. Two controlled results from 2026 sharpen what the rotation buys. A systematic comparison of KV-cache compression schemes finds that the rotation is worth nothing where the data is isotropic or low-rank — "the rotation neither helps nor hurts" — and everything where it is heavy-tailed, there being "the only viable scheme" [21]. The randomized Hadamard is therefore an insurance against tails, not a quality lever in general; the same paper reports that a fully random projection is uniformly worse than the structured one, and that a data-adaptive alternative (a per-head eigenbasis with water-filling [22]) is fragile — one spurious eigenvalue reverses the output — and pays off only at two to three bits and short sequences. For four-bit generation at 128K that comparison recommends the structured rotation, which is what this cache uses. The channel moments measured on this model say which regime it is in: the excess kurtosis before the rotation is −0.18, i.e. lighter than Gaussian, so the insurance is not needed here and the randomized variant is left out for a reason that can

be stated rather than assumed. One warning from the same work applies to this paper's own future steps: a residual correction on the *key* path is amplified exponentially by the softmax (Jensen's inequality), while a correction on the value path is not.

What this paper adds to that literature: the measurement. The published work above *observes* the per-channel structure that survives a Hadamard — a post-Hadamard mean-centering composed with the transform [10], a channel order chosen by a statistic measured after the rotation [9], a zero-point absorbing an outlier [18] — and treats it as a real property of trained models. What the literature does not publish is the *measurement* of where that property comes from, and this paper's own numerics supply it [32]: the bias is the data's and not the transform's (the un-rotated mean of the engine's own cells reproduces the channel-space mean at a ratio of 1.000 in every layer, the round trip exactly zero), the consequence is noise amplification rather than a shift, and the yardstick the comparison uses carries no drift of its own (the anchor's own channel-mean drift is three orders of magnitude below the bias, and its clamp never fires). A survey of the transform stage places the centering among the affine transforms that spend the last degrees of freedom [11]; the measurement says where those degrees should be spent here — not on a bias that the level table already removes.

8 Threats to Validity

The first threat is the obvious one: there is no 5.0-bit measurement. Every 5.0 number in this paper is either the byte model of Equation 1 (a linear model validated at one measured point, 5.289) or a linear interpolation between the two measured points (8.25/1,584 and 5.29/1,016). The model ignores per-layer variation, block-scale overhead beyond the 0.25 bit/cell assumed for `int8`, and any non-linearity in the ring's cost; the 960 MiB and 457-slot figures inherit all three. The three ring tiers are arithmetic: the `int8` and 12-bit tiers do not exist in the engine, and their bit costs are the default's and a nominal 12.0.

The second threat is the reference. "Not behind `int8`" is not "not behind fp16": `int8` carries its own quantization error, and a 5.0 arm that matches it inherits that error. The target is deliberately relative, because the product question is whether to keep `int8` as the quality default, not whether 4-bit can be lossless. A reader who wants an fp16-relative claim will not find one here, and the predecessor's KL values are all against the fp16 reference for exactly that reason.

The third threat is coverage. The quality evidence is one task family (RULER complex word extraction), scored teacher-forced, single-segment, on one model, one GPU, one operating system, one stack and one weight set. The target's "all areas" is therefore all *measured* areas; multi-turn growth, concurrency, mid-segment KL against the other arms and a second task family are unmeasured, and the plan does not claim them. The 128K results are one text each with adjacent blocks, so their t values are descriptive; the 4K results have eight segments and are the only place where the sign count is a real statistic.

The fourth threat is the bar itself. K1's upper bound and K5's 10% are choices, not measurements; K4's 5.0 is a product decision. The plan mitigates this by fixing all of them before the first run and by reporting the raw numbers either way, so a later disagreement can be about the threshold rather than about the data. What it cannot mitigate is that a target set by the authors of the arm is a target the authors can move; the pre-registration and the published kills are the only defence, and they are worth exactly as much as the discipline of the run.

9 Conclusion

The window's successor has a precise job: store at most 5.0 bit/cell — 39% below `int8`, 6% below the shipped window — and not be behind `int8` on the paired 4K mean, the 128K whole segment, p99 or the last 1,024 positions on either text, with prefill within 10% and the correctness gates green. The byte budget says the ring may spend 0.50 bit/cell: 4,096 cells in fp16, 7,943 with an `int8` ring, 5,461 at 12 bit, against 6,464 today. The measured evidence says the remaining error is in the bulk and immediately outside the window, that a larger fp16 ring is arithmetically closed and was already tried at 4K without effect, and that the tail mean cannot carry a target. The six levers and the five-stage ladder turn that into a finite experiment: one counter run, one debug statistic, at most three 4K arms and one 128K block, with kills that end the branch instead of widening it.

The plan's most likely outcomes are a negative result and a 6.0-bit fallback, in that order. That is not a weakness of the plan; it is what a pre-registered target looks like before the data exist. If the `int8` ring plus a better bulk reaches 5.0, the window is not improved but replaced: the same 128K context would cost about 960 MiB instead of 1,584, with no statistic behind `int8`. If it does not, the negative result is the deliverable, and the shipped window stays the honest memory point at 5.29.

The state after the first ladder step. One measurement is in, and it decides the codec. The 16-levels-on-both-sides codec with the exact-cell window reaches `int8`'s level on cwe-4k (paired +0.00002 nats, $t = -0.02$), while the shipped 8-level-value probe is +0.00402 behind, and the level split is worth +0.00400 nats against it (per-position [+0.00138, +0.00663], $t = +2.99$). The per-channel offset — the lever this paper is named after — turns out to be a *substitute* for the level table rather than an addition to it: +0.00677 nats on the coarse probe, +0.00218 with the window (not significant), and nothing on the 16/16 codec. The product reading on 4K is therefore the 16/16 codec with the window and without the calibration: no per-model artifact, no basis discipline in the snapshot, no extra runtime.

At 128K that reading does not hold, and the plan says so. The same arm is +0.00741 nats behind `int8` on the first 128K text ($t = +4.80$), wider on the p99, and the gap grows with

the context length rather than shrinking. The 5.0-bit target is therefore not reached by this arm on this corpus, and the reason is the bulk codec in the long context, not the overlay — the overlay is what buys the 5.29 against 8.25 bit per cell. What the measurements close is the codec axis: the level split is significant and worth +0.00400 nats, the per-channel offset is a substitute for it and not a product path, and the stock cache sits +0.00843 behind. What they leave open is the ring: an `int8` ring would halve the overlay's byte price and make the 5.0 mark arithmetically reachable, and that is a geometry question, not a codec one.

References

[1] M. Kronenberger. *Hardening the Exact-Cell KV Window for* `--kv q4lm8v`*: Design, Decisions, and a Measurement-First Validation.* In-repo, `docs/paper/KV-Window-Hardening.pdf`, version 1.11, 2026-11-11.

[2] M. Kronenberger. *Ansatz-Verbesserungen: KV-Cache-Quantisierung & Exact-Cell-Fenster* (working document, German). In-repo, `docs/paper/Ansatz_Improvements.md`, 2026-10-25.

[3] G. Xiao, Y. Tian, B. Chen, S. Han, M. Lewis. *Efficient Streaming Language Models with Attention Sinks.* arXiv:2309.17453, 2023 (ICLR 2024). Code: https://github.com/mit-han-lab/streaming-llm.

[4] Z. Liu, J. Yuan, H. Jin, S. Zhong, Z. Xu, V. Braverman, B. Chen, X. Hu. *KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache.* ICML 2024, arXiv:2402.02750. Code: https://github.com/jy-yuan/KIVI.

[5] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, A. Gholami. *KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization.* NeurIPS 2024, arXiv:2401.18079. Code: https://github.com/SqueezeAILab/KVQuant.

[6] H. Kang et al. *GEAR: An Efficient KV Cache Compression Recipe for Near-Lossless Generative LLM Inference.* arXiv:2403.05527, 2024.

[7] H. Duanmu, Z. Yuan, X. Li, J. Duan, X. Zhang, D. Lin. *SKVQ: Sliding-window Key and Value Cache Quantization for Large Language Models.* arXiv:2405.06219, 2024.

[8] R. Liu, H. Bai, H. Lin, Y. Li, H. Gao, Z. Xu, L. Hou, J. Yao, C. Yuan. *IntactKV: Improving Large Language Model Quantization by Keeping Pivot Tokens Intact.* ACL Findings 2024, arXiv:2403.01241.

[9] Z. Su, Z. Chen, W. Shen, H. Wei, L. Li, H. Yu, K. Yuan. *RotateKV: Accurate and Robust 2-Bit KV Cache Quantization for LLMs via Outlier-Aware Adaptive Rotations.* arXiv:2501.16383, 2025 (IJCAI 2025).

[10] G. Federici et al. *HadaNorm: Adaptive Channel Centering for Post-Hadamard Quantization.* 2025. (Cited via the transform survey below; observed in the survey's account: post-Hadamard channels still differ in mean, so a dynamic mean-centering is composed with the transform.)

[11] E. Jokar. *Transforms for LLM Quantization: The Great Inversion and Format Co-Design.* arXiv:2608.25188, 2026. (A survey of the transform stage: 200 works, 43 transform methods classified, and the inversion between allocation-flexible coding and the shared-scale quantizer a deployed kernel performs.)

[12] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, J. Hensman. *QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs.* arXiv:2404.00456, 2024.

[13] Z. Liu, C. Zhao, I. Fedorov, B. Soran, D. Choudhary, R. Krishnamoorthi, V. Chandra, Y. Tian, T. Blankevoort. *SpinQuant: LLM quantization with learned rotations.* ICLR 2025, arXiv:2405.16406.

[14] A. Tseng, J. Chee, Q. Sun, V. Kuleshov, and C. De Sa. *QuIP#: Even Better LLM Quantization with Hadamard Incoherence and Lattice Codebooks.* ICML 2024, arXiv:2402.04396.

[15] R. Ben-Basat, W. Kuszmaul, M. Mitzenmacher, A. Portnoy, and S. Vargaftik. *Quantizing With Randomized Hadamard Transforms: Efficient Heuristic Now Proven.* 2026.

[16] K. Shao, X. Huang, J. Chen, J. Shao, Y. Wang, J. Yu, Q. Li, W. Zhang, K.-T. Cheng, and C. Y. Tsui. *FlagRot: Selective Rotation for Low-Bit KV Caches.* arXiv:2611.03992, 2026.

[17] *Output-Aware Rotation for INT2 KV-Cache Quantization.* arXiv:2608.02691, 2026.

[18] *OffQ: Taming Structured Outliers in LLM Quantization by Offsetting.* arXiv:2606.07116, 2026.

[19] R. Ben-Basat, W. Kuszmaul, M. Mitzenmacher, A. Portnoy, and S. Vargaftik. *Quantizing With Randomized Hadamard Transforms: Efficient Heuristic Now Proven.* arXiv:2605.06014, 2026.

[20] *KV-cache mean-centering and its calibration-basis guard.* Public `llama.cpp` fork (PrismML-Eng), July 2026.

[21] P. D'Alberto, A. Sirasao, E. Delaye, and R. Patwari. *Ablation, Statistical Inference, and Validation for KV-Cache Compression.* arXiv:2607.09683, 2026.

[22] A. Gopinath. *3% Is All You Need: Breaking TurboQuant's Compression Limit via Spectral Structure.* Technical report, 2026.

[23] A. Zandieh et al. *TurboQuant: Online Vector Quantization with Near-Optimal Distortion Rate.* ICLR 2026, arXiv:2504.19874.

[24] C.-C. Chang, W.-C. Lin, C.-Y. Lin, C.-Y. Chen, Y.-F. Hu, P.-S. Wang, N.-C. Huang, L. Ceze, M. S. Abdelfattah, K.-C. Wu. *Palu: Compressing KV-Cache with Low-Rank Projection.* arXiv:2407.21118, 2024.

[25] J. Jia, J. Li, Z. Zhou, J. H. Heo, J. Wang, T. Dao, S. L. Song, B. Athiwaratkun, C. Xu, T. Zhang, X. Wu. *SAW-INT4: System-Aware 4-Bit KV-Cache Quantization for Real-World LLM Serving.* arXiv:2604.19157, 2026.

[26] M. Hariri, A. Luo, W. Chen, S. Zhong, T. Zhang, Q. Wang, X. Hu, X. Han, V. Chaudhary. *Quantize What Counts: More for Keys, Less for Values.* ACL 2026, arXiv:2502.15075.

[27] X. Li, Z. Xing, Y. Li, L. Qu, H. Zhen, Y. Yao, W. Liu, S. J. Pan, M. Yuan. *KVTuner: Sensitivity-Aware Layer-Wise Mixed-Precision KV Cache Quantization for Efficient and Nearly Lossless LLM Inference.* ICML 2025, arXiv:2502.04420. Code: https://github.com/cmd2001/KVTuner.

[28] J. Zhang et al. *More Tokens, Lower Precision: Towards the Optimal Token-Precision Trade-off in KV Cache Compression.* arXiv:2412.12706, 2024.

[29] ggml-org, `llama.cpp` repository: KV-cache quantization types (`-cache-type-k/v`: fp16, q8_0, q4_0). https://github.com/ggml-org/llama.cpp.

[30] J. Kübler, E. Kurtić, et al. (vLLM team), *The State of FP8 KV-Cache and Attention Quantization in vLLM*, vLLM blog, 2026-04-22. https://vllm-project.github.io/2026/04/22/fp8-kvcache.html.

[31] Strata, `docs/DETAILS.md`: KV-cache quantization section (byte model, KL measurements, window 256/64 at 128K on both 128K texts). In-repo.

[32] Strata, `tools/kv_bias_origin.py`, `tools/kv_int8_drift.py` (with `bench/results/2026-11-10-centering/`): the numerical study of the per-channel bias of a rotated cache — the bias is the data's and not the transform's (the un-rotated dump mean reproduces the probe's channel-space mean at a ratio of 1.000 in every layer and every head, the round trip $H(H(y)) - y$ is exactly zero), and the reference codec carries no drift of its own (the int8 KV codec's channel-mean drift is $1.2 \cdot 10^{-4}$ sd against the data's own 0.2673 sd; its clamp fires in none of 402 million values). In-repo, reproducible.

[33] Strata, `tools/kv_codec_mse.py`, `tools/kv_center_attrib.py` (with `bench/results/2026-11-10-merge-reeval/`): the block-level codec comparison on the engine's own cells, split by layer and by side, with the two centering modes named (the offsets the engine uses, and the naive one whose number the campaign's first reading rests on). In-repo, reproducible.

[34] Strata, `docs/QUANTS.md`: quant-quality yardsticks (mean KL, KL p99, top-1, perplexity) for the weight-quantization experiment; the cwe KL protocol is documented in the research-branch harness. In-repo.

[35] Strata, `bench/results/2026-09-27-kv-q4`: Q4_0 KV vs fp16/int8 document perplexity at 1K/8K (Q2_0). In-repo.

[36] Strata, `docs/KV_QUANT_WORKPACKAGE.md` (branch `KV_QUANT_research`): workpackage report, window history §7.12–§7.21, K-versus-V falsification, crossover withdrawal. In-repo.

[37] Strata, `bench/results/2026-10-24-needle-window`: same-day needle pair, engine 0.1.47, `int8` against the 256/64 window, 6/6 and 6/6. In-repo.